Onyx — PrinterKit reference

Onyx · multi-process OS on Raspberry Pi 4

*The reference of **PrinterKit** (`SD:/lib/printerkit.so`, `user/Kits/printerkit`): what it is for, how a program uses it, and every operation it exposes. The operations' part is made from the kit's headers by `tools/docgen/kitdocs.py` — the headers are the source. Overview of all the kits: The Kits.*

Contents

1. What it is
2. Using it
3. Index
4. `printerkit/printerkit.h`

What it is

PrinterKit is printing: the Print dialog, a job and its pages — text, lines, shapes, pictures — handed to the print service, which makes of them what the printer takes, or a PDF file.

Include	`#include "printerkit/printerkit.h"`
Link	`lib/printerkit.imp.a`
Library	`SD:/lib/printerkit.so` — 33 entries in its table (`user/Kits/printerkit/printerkit.abi`, append-only)
Sources	`user/Kits/printerkit/`

Using it

A program prints in four steps: the Print dialog, a job, its pages, the end. Lengths are points (1/72 inch) from the page's top-left corner; colours are `0xRRGGBB`. The pages are recorded: the print service turns them into what the printer takes — or into a PDF file.

```
#include "printerkit/printerkit.h"

PrintSetup s;
print_setup_default (&s);
PrintDialogInfo di = { sizeof di, "My document", 1, 0, 0, 0, 0 };
if (print_dialog (&s, &di))                          // the printer, the paper, the pages
{
    PrintJob *j = print_begin (&s, "My document");
    int font = print_font (j, "DejaVu Sans", 0);
    if (print_page (j, 0, 0))                        // 0, 0: the paper chosen
    {
        print_text (j, font, 12, 72, 84, "Hello, printer", 0x000000);
        print_line (j, 72, 90, s.paper_w - 72, 90, 0.75f, 0x808080);
    }
```

```
    print_end (j);                                  // to the print queue
}
```

Rectangles, paths (lines and curves, filled or stroked) and pictures are drawn the same way (`print_rect`, `print_path_*`, `print_image`).

Index

Everything the headers declare, in their order — the details are in each header's part below.

Name	What it does	Header
`PrintSetup`	What the user chose in the Print dialog.	`printerkit.h`
`PrintDialogInfo`	the page is fitted on the printer's paper	`printerkit.h`
`print_setup_default`	the default printer, its default paper, upright, one copy	`printerkit.h`
`print_dialog`	1: print (s filled), 0: cancelled	`printerkit.h`
`print_setup_paper`	another paper -> 1, 0 unknown name	`printerkit.h`
`print_begin`	0: the spool cannot be written	`printerkit.h`
`print_page`	a new page (w, h 0: the setup's paper) -> 1, or 0: outside	`printerkit.h`
`print_rect`	the pages to print -- skip its drawing (it would be ignored)	`printerkit.h`
`print_line`	the pages to print -- skip its drawing (it would be ignored)	`printerkit.h`
`print_frame`	a rectangle's outline	`printerkit.h`
`print_path_move`	a shape	`printerkit.h`
`print_path_line`	a shape	`printerkit.h`
`print_path_curve`	a shape	`printerkit.h`
`print_path_close`	a shape	`printerkit.h`
`print_path_fill`	a shape	`printerkit.h`
`print_path_stroke`	a shape	`printerkit.h`
`print_image`	an image	`printerkit.h`
`print_font`	text	`printerkit.h`
`print_text`	text	`printerkit.h`
`print_text_width`	text	`printerkit.h`

Name	What it does	Header
`print_font_metrics`	text	`printerkit.h`
`print_font_data`	for an app with its own text layout	`printerkit.h`
`print_glyph`	for an app with its own text layout	`printerkit.h`
`print_end`	the job queued -> its number (> 0), or < 0: it could not be	`printerkit.h`
`print_abort`	nothing printed	`printerkit.h`
`PrintPrinter`	(a type)	`printerkit.h`
`print_printers`	-> how many	`printerkit.h`
`print_printer_media`	a paper of a printer	`printerkit.h`
`print_printer_add`	1 / 0 (err: why)	`printerkit.h`
`print_printers_find`	the printers found on the network into out (max, 16 at most) -> how many (0: none, no print service)	`printerkit.h`
`print_printer_remove`	taken off the list, by its name -> 1 / 0 (not known; the PDF printer stays)	`printerkit.h`
`print_printer_default`	made the default printer, by its name -> 1 / 0 (not known)	`printerkit.h`
`print_printer_status`	asked now: 1 ready (text: ink...), 0 (text: why)	`printerkit.h`
`print_jobs`	-> how many (the queue, then the last finished)	`printerkit.h`
`print_job_cancel`	a waiting or printing job stopped -> 1 / 0 (no such job, no print service)	`printerkit.h`
`print_jobs_forget`	the finished jobs dropped from the list	`printerkit.h`

`printerkit/printerkit.h`

printerkit/printerkit.h -- printing from an app (the shared library SD:/lib/printerkit.so; docs/03 "Printing").

An app prints in four steps -- the Print dialog, a job, its pages, the end:

```
  PrintSetup s; print_setup_default (&s);
  PrintDialogInfo di = { sizeof di, "My document", pageCount, 0, 0, 0, 0 };
```

```
  if (print_dialog (&s, &di))                              // the user chose a printer, a paper,
the pages...
  {
      PrintJob *j = print_begin (&s, "My document");
      int f = print_font (j, "DejaVu Sans", 0);
      for (int p = 0; p < pageCount; p++)
          if (print_page (j, 0, 0))                        // 0, 0: the paper chosen; 0
returned: not in the range
          {
              print_text (j, f, 12, 72, 72 + 12, "Hello", 0x000000);
              print_line (j, 72, 90, s.paper_w - 72, 90, 0.75f, 0x808080);
          }
      print_end (j);                                       // the job goes to printd's queue
  }
```

Lengths are points (1/72 inch), y goes down from the page's top-left corner, colours are 0xRRGGBB. The pages are recorded, not rendered: the print service (printd) makes of them what the printer takes -- a PDF file for the PDF printer, pages of pixels at its resolution for a network printer -- and tells the user when the job is done. The app does not wait for the printer.

A program links lib/printerkit.imp.a (with lib/uikit.imp.a and lib/fontkit.imp.a: the dialog is uikit's, the text FreeType's). The table is append-only (printerkit/printerkit.abi): these functions never change.

MIT License -- Copyright (c) 2026 Stéphane Wegener and the Onyx contributors. Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the "Software"), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions: The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software. THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED.

```
typedef struct PrintJob PrintJob;

enum { PRINT_DRAFT = 3, PRINT_NORMAL = 4, PRINT_HIGH = 5 };     // quality
enum { PRINT_PORTRAIT = 0, PRINT_LANDSCAPE = 1 };
enum { PRINT_BOLD = 1, PRINT_ITALIC = 2 };                      // a font's style
enum { PRINT_IMG_ALPHA = 1,     // the pixels are 0xAARRGGBB (else 0x00RRGGBB: opaque)
       PRINT_IMG_PHOTO = 2 };       // a photo: kept as a JPEG (smaller; not for text or line
art)
```

What the user chose in the Print dialog. Fields are only ever added at the end (size says how many).

```
typedef struct PrintSetup
{
    unsigned size;              // sizeof (PrintSetup)
    char  printer[64];          // the printer's name ("PDF": the PDF printer)
    char  media[64];            // the paper, as IPP names it ("iso_a4_210x297mm")
    float paper_w, paper_h;         // the page in points, the orientation applied
    float margin_l, margin_t, margin_r, margin_b;   // what the printer cannot print on,
along each edge
    int   orientation;          // PRINT_PORTRAIT / PRINT_LANDSCAPE
    int   copies;
    int   mono;             // 1: black and white
    int   quality;              // PRINT_DRAFT / NORMAL / HIGH
    int   from, to;             // the pages to print, from 1; 0, 0: all
    char  output[256];          // the PDF printer: the file to write
```

```
    int   reserved[16];
} PrintSetup;

enum { PRINT_DLG_OWN_PAPER = 1 };   // the document has its own page size (a slide, a PDF):
no paper choice --
```

the page is fitted on the printer's paper

```
typedef struct PrintDialogInfo
{
    unsigned size;              // sizeof (PrintDialogInfo)
    const char *title;          // the document's name (the dialog's title, the PDF's file
name)
    int   pages;                // how many pages it has (0: not known)
    int   current;              // the page shown (from 1: offers "Current page"; 0: no such
choice)
    int   flags;                // PRINT_DLG_*
    float paper_w, paper_h;         // the document's own paper to select first (0: the
printer's default)
} PrintDialogInfo;
```

the dialog

```
void print_setup_default (PrintSetup *s);           // the default printer, its default
paper, upright, one copy
int  print_dialog (PrintSetup *s, const PrintDialogInfo *info); // 1: print (s filled), 0:
cancelled
int  print_setup_paper (PrintSetup *s, const char *media, int orientation); // another paper
-> 1, 0 unknown name
```

a job

```
PrintJob *print_begin (const PrintSetup *s, const char *title); // 0: the spool cannot be
written
int   print_page (PrintJob *j, float w, float h);   // a new page (w, h 0: the setup's paper)
-> 1, or 0: outside
```

the pages to print -- skip its drawing (it would be ignored)

```
void  print_rect (PrintJob *j, float x, float y, float w, float h, unsigned rgb);
void  print_line (PrintJob *j, float x0, float y0, float x1, float y1, float width, unsigned
rgb);
void  print_frame (PrintJob *j, float x, float y, float w, float h, float width, unsigned
rgb); // a rectangle's outline
```

a shape: move to, lines and Bezier curves, closed or not; then filled, or stroked with a line

```
void  print_path_move (PrintJob *j, float x, float y);
void  print_path_line (PrintJob *j, float x, float y);
void  print_path_curve (PrintJob *j, float x1, float y1, float x2, float y2, float x3, float
y3);
void  print_path_close (PrintJob *j);
void  print_path_fill (PrintJob *j, unsigned rgb);
void  print_path_stroke (PrintJob *j, float width, unsigned rgb);
```

an image: pw x ph pixels drawn in (x, y, w, h)

```
void  print_image (PrintJob *j, const unsigned *px, int pw, int ph, float x, float y, float
w, float h, int flags);
```

text: a font by its family ("DejaVu Sans", "Liberation Serif"...: SD:/res/fonts, SD:/fonts) and style -> its number (-1: no such font nor a fallback); the text at its baseline -> the pen's advance

```
int   print_font (PrintJob *j, const char *family, int style);
float print_text (PrintJob *j, int font, float size, float x, float baseline, const char
*utf8, unsigned rgb);
float print_text_width (PrintJob *j, int font, float size, const char *utf8);
void  print_font_metrics (PrintJob *j, int font, float size, float *ascent, float *descent,
float *line);
```

for an app with its own text layout: a TrueType file's bytes (kept by the app until print_end) -> the font's number; one glyph of it at its pen position (style: PRINT_BOLD / ITALIC made from this face)

```
int   print_font_data (PrintJob *j, const unsigned char *ttf, unsigned len);
void  print_glyph (PrintJob *j, int font, float size, float x, float baseline, unsigned
glyph, unsigned unicode, unsigned rgb, int style);
int   print_end (PrintJob *j);      // the job queued -> its number (> 0), or < 0: it could
not be
void  print_abort (PrintJob *j);    // nothing printed
```

the printers and the queue (the Control Panel's applet; an app's own status)

```
typedef struct PrintPrinter
{
    char name[64], kind[8], uri[160], model[64];    // kind: "pdf", "ipp"
    int  color, copies, dpi, is_default;
    unsigned quality;               // bits 3 (draft), 4 (normal), 5 (high)
    char media_default[64];
} PrintPrinter;
enum { PRINT_JOB_QUEUED = 0, PRINT_JOB_PREPARING, PRINT_JOB_SENDING, PRINT_JOB_PRINTING,
PRINT_JOB_DONE, PRINT_JOB_FAILED, PRINT_JOB_CANCELED };
typedef struct PrintJobInfo { unsigned id; int state, pages, page; char printer[64],
title[80], message[96]; } PrintJobInfo;

int  print_printers (PrintPrinter *out, int max);       // -> how many
```

a paper of a printer: its IPP name, its name for a person -> 1, 0: no i-th paper

```
int  print_printer_media (const char *printer, int i, char *name, int ncap, char *label, int
lcap);
```

a network printer asked what it can do, and added (address: "192.168.0.14", "ipp://host:631/ipp/print")

```
int  print_printer_add (const char *name, const char *address, char *err, int cap); // 1 / 0
(err: why)
```

the local network searched for printers (mDNS / DNS-SD, as AirPrint: a few seconds) -> how many; usable 0: it answers but takes neither PWG Raster nor PDF

```
typedef struct PrintFound { char address[32], name[64], model[64]; int usable; } PrintFound;
int  print_printers_find (PrintFound *out, int max);    // the printers found on the network
into out (max, 16 at most) -> how many (0: none, no print service)
int  print_printer_remove (const char *name);   // taken off the list, by its name -> 1 / 0
(not known; the PDF printer stays)
int  print_printer_default (const char *name);  // made the default printer, by its name -> 1
/ 0 (not known)
int  print_printer_status (const char *name, char *text, int cap);  // asked now: 1 ready
(text: ink...), 0 (text: why)
int  print_jobs (PrintJobInfo *out, int max);           // -> how many (the queue, then the
last finished)
```

```
int  print_job_cancel (unsigned id);    // a waiting or printing job stopped -> 1 / 0 (no
such job, no print service)
void print_jobs_forget (void);                  // the finished jobs dropped from the list

}
#endif
```